

(TR-104) PYTHON WITH AI/ML

Training Day 40 Report

Date: 6 March 2026

Topic Covered: FastAPI Integration, Centralized Configuration & Optimization of RAG-Based AI Assistant

The fortieth day of training focused on improving the architecture, logging, configuration management, and API integration of the RAG-based AI Assistant project for better maintainability, performance, and production readiness.

- Improved all source files with proper logging and structured error handling mechanisms.
- Fixed duplicate guardrail calls inside `rag_pipeline.py` for optimized execution flow.
- Resolved duplicate log generation issues in retriever and RAG pipeline modules.
- Updated the local LLM from Mistral to Llama 3.2 using Ollama.
- Created centralized configuration management using `pydantic-settings`.
- Updated `ChromaRetriever` to use centralized settings for vector database collection and storage paths.
- Merged `search_similar_documents` functionality into `ChromaRetriever` for faster retrieval performance.
- Disabled embedding progress bar to improve response speed and reduce unnecessary console output.
- Created a centralized `logger.py` module for reusable logging across the project.
- Implemented FastAPI endpoint integration for the RAG pipeline.
- Tested query and response flow through the API service.
- Created a structured `README.md` file containing proper project documentation and folder structure explanation.
- Strengthened understanding of scalable RAG architecture, centralized configuration handling, API integration, and production-ready AI system design.

- Learned optimization practices for retrieval pipelines, logging systems, and local LLM integration workflows.
- **GitHub Link:**
<https://github.com/JaspinderKaurWalia26/projects>
- **Blog Link:**
<https://medium.com/@jaspinderkaurwalia855/building-a-rag-system-from-scratch-with-fastapi-chromadb-and-llama-3-2-f3d3695950eb>